

Linear Algebra

LearnOpenGL recommends [GLM](#), and some of your other undergraduate courses may recommend [Eigen](#). In CS171, though, we'll be using a rather obscure math library called [Arietta](#) for all upcoming OpenGL assignments. Before you start wondering whether your TAs have lost their minds, we promise we've thought this through (see [Appendix A](#) for the reasons). As you get more comfortable with graphics and C++, this choice may start to make more sense.

This document introduces you to Arietta. We strongly recommend opening `Main.cpp` and typing out the examples yourself as you work through the tutorial.

1. Construction

Here's how to construct a vector:

```
Vec v0{1, 2}; // Creates a 2x1 column vector; the element type is deduced as int.
Vec v1{1.0F, 2.0F, 3.0F}; // Creates a 3x1 column vector; the element type is float.
```

These forms are equivalent:

```
Mat v0{1, 2}; // Writing `Mat` here has exactly the same effect as writing `Vec`.
Vec2 v1{1, 2}; // Explicitly specifies 2 rows; otherwise equivalent.
Vec2i v2{1, 2}; // Explicitly specifies the element type as int; otherwise equivalent.
```

Besides `i` (`int`), the available type suffixes are `u` (`unsigned int`), `f` (`float`), and `d` (`double`).

The following forms, however, are not allowed. We'll explain why in [Section 4](#):

```
Vec<int, 2> v0{1, 2}; // Does not compile.
Vec2<int> v1{1, 2}; // Does not compile.
```

To construct a matrix, put its column vectors together:

```
Mat m0{Vec{0, 1}, Vec{2, 3}}; // The first column is {0, 1}; the second is {2, 3}.
```

You can also write:

```
// Explicitly specifies 2 rows and 2 columns; otherwise equivalent.
Mat2 m0{Vec{0, 1}, Vec{2, 3}};
// Explicitly specifies the element type as int; otherwise equivalent.
Mat2i m1{Vec{0, 1}, Vec{2, 3}};
```

2. Constants

Here's how to construct a vector whose elements all have the same constant value:

```
// Creates a 2x1 column vector of ints, with every element equal to 1.
auto v0 = Vec<int, 2>::Constant<1>();
// Creates a 3x1 column vector of floats, with every element equal to 2.
auto v1 = Vec<float, 3>::Constant<2.0F>();
```

These two forms are equivalent to the first example above:

```
auto v0 = Vec2<int>::Constant<1>(); // Equivalent.
auto v1 = Vec2i<>::Constant<1>(); // Equivalent.
```

There are a few other functions for creating constants:

```
// Creates a 2x1 column vector of ints, with every element equal to 0.
auto v0 = Vec2i<>::Zero();
// Creates a 3x3 identity matrix with float elements.
auto v1 = Mat3f<>::Identity();
```

3. Operations

Use `operator()` to access the elements of a matrix or vector:

```
Vec vec{1.0F, 2.0F, 3.0F};
auto x = vec(0); // Row 1: 1.0F
auto y = vec(1); // Row 2: 2.0F
auto z = vec(2); // Row 3: 3.0F

Mat mat{Vec{0, 1}, Vec{2, 3}};
auto v00 = mat(0, 0); // Row 1, column 1: 0
auto v10 = mat(1, 0); // Row 2, column 1: 1
auto v01 = mat(0, 1); // Row 1, column 2: 2
auto v11 = mat(1, 1); // Row 2, column 2: 3
```

Arietta does not support accessors such as `.x()`, `.y()`, or `.z()`, or `operator[]`, and there are no plans to add them. All element access goes through `operator()`. This design requires code to use the same calling syntax as functors and lambda functions, keeping it compatible with both.

The supported operators are `+`, `-`, `*`, `/`, `+=`, `-=`, `*=`, `/=`, `==`, and `!=`:

```
Mat mat{Vec{0, 1}, Vec{2, 3}};
Vec vec{4, 5};
auto res = mat * vec;
```

The available methods are `Dot`, `Cross`, `Norm2`, `Norm`, `Transpose`, `Determinant`, and `Inverse`. Here, `Norm2` means the squared norm. Here's an example of how to use these methods:

```
Vec v0{2, 0, 0};
Vec v1{0, 3, 0};
auto res = v0.Cross(v1); // Produces the vector {0, 0, 6}.
```

You can also convert the element type of a matrix or vector:

```
Vec v0{1, 2, 3};
auto v1 = value_type::Cast<float>(v0); // Produces the vector {1.0, 2.0, 3.0}.
```

4. Advanced

In computer graphics, we often work with matrices and vectors where some elements already have known values. For example:

- In homogeneous coordinates, we usually represent a 3D point as $(x, y, z, 1)$ and a direction as $(x, y, z, 0)$.
- In a scaling matrix, only the diagonal entries can be nonzero; all other entries are 0.
- And there are plenty more examples...

So what does that have to do with our code? Let's look at this example:

```
float x = 2.0F, y = 3.0F;
Vec v0{x, 0.0F, 0.0F};
Vec v1{0.0F, y, 0.0F};
auto res = v0.Cross(v1); // Produces the vector {0, 0, x * y}.
```

The last line calls `Cross` to compute a cross product, which normally takes several multiplications and additions or subtractions. But a quick bit of mental math tells us that the result must be $\{0, 0, x * y\}$. With all those zeros in `v0` and `v1`, the whole calculation boils down to a single multiplication. The compiler, however, may not work this out, so it might not simplify the cross product for us.

Arietta requires all constructor arguments to have the same underlying element type. Since `x` and `y` are `float`, we use `0.0F` rather than `0`. The compile-time zero in the next example must also have a `float` value.

Arietta provides a mechanism that guarantees this simplification happens at compile time, leaving just one multiplication. All we need to do is rewrite the code like this:

```
float x = 2.0F, y = 3.0F;
// Defines a compile-time zero that tells compilers to go all in on simplification.
C<0.0F> zero{};

Vec v0{x, zero, zero};
Vec v1{zero, y, zero};
auto res = v0.Cross(v1); // Produces the vector {zero, zero, x * y}.
```

Continuing with this example, we can even check at compile time that the first two elements of `res` are zero. Use compile-time indices to access them:

```
static_assert(res(C<0>{}) == 0.0F);
static_assert(res(C<1>{}) == 0.0F);
```

Going a step further, we can assert that `v0`, `v1`, and `res` each take up only as much space as a single `float`:

```
static_assert(sizeof(v0) == sizeof(float));
static_assert(sizeof(v1) == sizeof(float));
static_assert(sizeof(res) == sizeof(float));
```

In other words, Arietta lets us define matrices and vectors with a mix of runtime and compile-time elements. It supports algebraic operations on these special matrices just as it does on ordinary ones. This is the main reason we chose it as the math library for CS171.

That sounds great, but when storing a matrix or vector as a class member, we usually want a more general storage type: one where every element is a runtime value. Arietta provides `AsStorage` for this purpose. Continuing with the same `v0`, `v1`, and `res`:

```
struct SomeStruct {
    Vec3f<>::AsStorage member;
} s;

// Stores it as the vector {0, 0, x * y}, with all elements stored at runtime.
s.member = res;

s.member = v0.Cross(v1); // You can also assign the result directly.
```

That's why you'll occasionally see `AsStorage` in the assignments, especially in class member declarations.

A. About Linear Algebra Libraries

As TAs who work on graphics research every day, we've had our fair share of frustrations with math libraries:

- Eigen: Arguably the most powerful math library out there, but its implementation is tightly tied to CPU computation. This makes it a poor fit for newer C++ features and CUDA (GPU programming); it still doesn't fully support `constexpr`.
- GLM: A long-established library with plenty of features that's quite pleasant to use. Its structure has grown complex over the years, though, and it doesn't support storing `constexpr` constants in CUDA's `__constant__` memory.
- linalg: Offers roughly the same set of features as Arietta, but likewise doesn't support storing `constexpr` constants in CUDA's `__constant__` memory.